Online Training Program
- ASP.NET Core Web API Online Training

ASP.NET WEB API – Basics
- Introduction to ASP.NET Web API
- Creating ASP.NET Web API Application
- How to add Swagger in Web API
- How to use Fiddler to test Web API
- How to use POSTMAN to test Web API
- ASP.NET Web API using SQL Server
- Content Negotiation in Web API
- Media Type Formatter in Web API
- How to Implement GET Method in Web API
- How to Implement POST Method in Web API
- How to Implement PUT Method in Web API
- How to Implement DELETE Method in Web API
- Custom Method Names in Web API
- Parameter Binding in Web API
- Consuming Web API Service From jQuery
- Calling Web API Service in a Cross Domain Using jQuery AJAX
- Cross-Origin Resource Sharing in WEB API

WEB API Routing and Attribute Routing
- ASP.NET Web API Routing
- **Routing Variations in WEB API**
- Routing and Action Selection in Web API
- ASP.NET Web API Attribute Routing
- Optional Parameters in Web API Attribute Routing
- Route Prefix in Web API
- Web API Attribute Routing Route Constraints
- Route Names and Route Orders in Attribute Routing

ASP.NET WEB API – Security
- Authentication and Authorization in ASP.NET Web API
- ASP.NET Web API Basic Authentication
- Role-Based Basic Authentication in Web API
- Web API Service with Basic Authentication
- HTTP Message Handlers in WEB API
- Basic Authentication Using Message Handler in Web API
- HTTP Client Message Handler in Web API
- Token Based Authentication in ASP.NET Web API
- Client Validation Using Basic Authentication in Web API
- Refresh Token in Web API
- Consume Refresh Token in C# Client
- HMAC Authentication in Web API

Web API Versioning
- Web API Versioning Using URI
- Web API Versioning using Query string Parameter
- Web API Versioning Using Custom Header
- Web API Versioning Using Accept Header
- Web API Versioning Using Custom Media Types

Advanced Concepts
- Enable SSL in Visual Studio Development Server
- Enable HTTPS in Web API

Popular ASP.NET Web API Books
- Most Recommended ASP.NET Web API Books

Routing Variations in WEB API

Back to: ASP.NET Web API Tutorials For Beginners and Professionals

Routing Variations in ASP.NET Web API

In this article, I am going to discuss **Routing Variations in ASP.NET WEB API** with examples. Please read our previous article before proceeding to this article where we discussed the basics of **Routing in ASP.NET Web API**. As part of this article, we are going to discuss the following important pointers.

1. **Understanding HTTP Methods.**
2. **Routing Variation in Web API by Action Name.**
3. **Understanding the NonAction attribute in ASP.NET Web API.**

Understanding HTTP Verbs in ASP.NET Web API.

In the last article, we discussed using the default naming convention i.e. default route provided by ASP.NET Web API Framework. Instead of using the default naming convention for the HTTP methods, you can also explicitly specify the **HTTP method** for action by decorating the action method with the **HttpGet, HttpPut, HttpPost, or HttpDelete** attribute.

Example: HttpGet, HttpPut, HttpPost, or HttpDelete Attribute in ASP.NET MVC

Let us understand this with an example. Please consider the following Student controller.

```
public class StudentController : ApiController
{
    [HttpGet]
    public IEnumerable<Student> FindAllStudents()...
    [HttpGet]
    public Student FindStudentById(int id)...
    [HttpDelete]
    public HttpResponseMessage RemoveStudent(int id)...
    [HttpPost]
    public HttpResponseMessage AddStudent(Student student)...
}
```

In the above example, the **FindAllStudents** and **FindStudentById** actions are mapped to the GET request, while the **RemoveStudent** action is mapped to DELETE Request and the **AddStudent** action is mapped to POST Request. If you want to allow multiple HTTP verbs on a single action method, or if you to allow HTTP methods other than the GET, PUT, POST, and DELETE, then you need to use the **AcceptVerbs** attribute, which takes a list of HTTP methods as shown in the below image.

```
public class StudentController : ApiController
{
    [AcceptVerbs("GET", "HEAD")]
    public Student FindStudentById(int id)...
}
```

In the above example, the FindStudentById action method is mapped to both GET and HEAD HTTP Request.

Routing Variation in Web API by Action Name

With the default routing template, the ASP.NET Web API Framework uses the HTTP method to select the action. However, if you want you can change this default behavior. Suppose, you want to create your own route where the action name is included as part of the URI, Then you need to create a new route as shown in the below image.

```
config.Routes.MapHttpRoute(
    name: "ActionApi",
    routeTemplate: "api/{controller}/{action}/{id}",
    defaults: new { id = RouteParameter.Optional }
);
```

In the above route template, the **{action}** parameter names the action method on the controller. With this style of routing, it is mandatory to use the HTTP verb attributes to specify the allowed HTTP methods. Let us understand this with an example. Please consider the following controller.

```
public class StudentController : ApiController
{
    [HttpGet]
    public IEnumerable<Student> FindAllStudents()...
    [HttpGet]
    public Student FindStudentById(int id)...
    [HttpDelete]
    public HttpResponseMessage RemoveStudent(int id)...
    [HttpPost]
    public HttpResponseMessage AddStudent(Student student)...
}
```

In the above example, a GET request for "**api/Student/FindAllStudents**" would map to the **FindAllStudents** action method. In ASP.NET Web API by using the **ActionName** attribute you can also override the action method name. In the below Student Controller class, we have two actions which map to "**api/Student/Image**". One action method supports the GET request while the other one supports the POST HTTP request.

```
public class StudentController : ApiController
{
    [HttpGet]
    [ActionName("Image")]
    public IEnumerable<Student> FetchStudentImage()...

    [HttpPost]
    [ActionName("Image")]
    public HttpResponseMessage AddStudentImage(Student student)...
}
```

Understanding the NonAction Attribute in ASP.NET Web API:

If you want to prevent an action method from getting invoked as a response to an HTTP request, then you need to decorate that action method with the **NonAction** attribute. This tells the ASP.NET Web API Framework that the method is not an action, even if it would match the routing rules.

```
public class StudentController : ApiController
{
    [NonAction]
    public IEnumerable<Student> SomePrivateMethid()...
}
```

As of now, we have discussed a high-level overview of ASP.NET Web API Routing. In the next article, I am going to discuss **how the WEB API Framework matches a URI to a route, selects a controller, and then selects the action to invoke**. Here, in this article, I try to explain **Routing Variations in WEB API** step by step with some examples. I hope this Routing Variation in WEB API article will help you with your need.

Dot Net Tutorials
About the Author: Pranaya Rout

Pranaya Rout has published more than 3,000 articles in his 11-year career. Pranaya Rout has very good experience with Microsoft Technologies, Including C#, VB, ASP.NET MVC, ASP.NET Web API, EF, EF Core, ADO.NET, LINQ, SQL Server, MYSQL, Oracle, ASP.NET Core, Cloud Computing, Microservices, Design Patterns and still learning new technologies.

1 thought on "Routing Variations in WEB API"

BENJAMIN
JULY 29, 2021 AT 10:42 AM

Very useful, thanks.

Reply

Leave a Reply

Your email address will not be published. Required fields are marked *

Comment *

Name* | Email* | Website

Post Comment

GET IT ON Google Play
© Dot Net Tutorials | Website Design by Sunrise Pixel